Person Re-Identification:

Reproduction and Analysis of TransReID (ICCV 2021) and CAL (CVPR 2022)

Research report

Standard and clothes-changing person re-identification on Market-1501 and PRCC

Prepared by: [Your name]

Submitted to: [Supervisor name]

October 2026

Abstract

Person re-identification (ReID) is the task of finding images of the same person across cameras that do not overlap. This report studies two methods published at top computer vision conferences. **TransReID** (ICCV 2021) is the first pure transformer method for object ReID and targets the standard setting, in which a person wears the same clothes in every camera. **CAL**, the Clothes-based Adversarial Loss (CVPR 2022), targets the harder clothes-changing setting, in which the same person appears in different outfits, and it uses RGB images only.

Both methods were re-implemented in PyTorch following the authors' official code and configuration files, and trained from ImageNet pre-trained weights on Kaggle (two NVIDIA Tesla T4 GPUs). The datasets were Market-1501 for TransReID and PRCC for CAL.

The reproduction closely matches the published results. TransReID reaches **95.07% Rank-1 and 88.89% mAP** on Market-1501, compared with 95.2% and 88.9% in the paper. CAL reaches **53.26% Rank-1 and 54.93% mAP** in the PRCC clothes-changing setting, compared with 55.2% and 55.8%, and 100% Rank-1 in the same-clothes setting. Our baseline without CAL also matches the baseline reported in the CAL paper, which confirms that the data pipeline and evaluation protocol are correct.

To connect the two papers, TransReID was also trained and tested on PRCC. It is almost perfect when clothes do not change (99.97% Rank-1) but falls to 48.60% Rank-1 and 50.41% mAP when they do. This is above the ResNet-50 baseline (45.70% and 45.75%) but clearly below CAL. A stronger backbone alone therefore gives only a small gain under clothes changes, while the training objective of CAL gives a much larger one.

Contents

Abstract 2
Contents 2
1 Introduction 4
1.1 Person re-identification 4
1.2 Clothes-changing person re-identification 4
1.3 Objectives of this study 4
1.4 Summary of findings 4
1.5 Structure of the report 4
2 Background 6
2.1 Problem formulation 6
2.2 Evaluation metrics 6
2.3 Common building blocks 6
3 Datasets 7
3.1 Market-1501 7
3.2 PRCC 7
3.3 LTCC 7
3.4 Evaluation protocols 8

4 Paper 1: TransReID (ICCV 2021) 9
4.1 Motivation 9
4.2 Transformer baseline 9
4.3 Side Information Embedding (SIE) 9
4.4 Jigsaw Patch Module (JPM) 9
4.5 Training objective 10
5 Paper 2: CAL (CVPR 2022) 11
5.1 Motivation 11
5.2 Clothes labels 11
5.3 Clothes classifier 11
5.4 Clothes-based adversarial loss 11
5.5 Training procedure 11
6 Experimental Setup 13
6.1 Hardware and software 13
6.2 Implementation 13
6.3 Differences from the official implementations 13
6.4 Experiments 13
7 Results 15
7.1 TransReID on Market-1501 15
7.2 CAL on PRCC 15
7.3 Effect of the clothes-based adversarial loss 16
7.4 Comparison of the two papers on clothes-changing data 16
7.5 Computational cost 17
7.6 Qualitative results 18
8 Discussion 19
8.1 Reproducibility 19
8.2 Why a strong standard model fails when clothes change 19
8.3 Practical considerations 19
8.4 Limitations 19
9 Conclusion 20
References 21
Appendix A: Evaluation Logs 22
Appendix B: Reproducing the Experiments 24

1 Introduction

1.1 Person re-identification

Given an image of a person (the *query*), person re-identification ranks a large set of images taken by other cameras (the *gallery*) so that images of the same person appear first. ReID is a core component of multi-camera tracking, crowd analysis and smart-city video systems. It is difficult because the same person can look very different across cameras because of viewpoint, body pose, lighting, image resolution, occlusion and background. At the same time, different people can look very similar, for example when they wear similar clothes.

Modern ReID systems learn an embedding network: each image is mapped to a feature vector, and images are compared by the distance between their vectors. Training uses a set of identities that is disjoint from the test identities, so the model must generalise to people it has never seen.

1.2 Clothes-changing person re-identification

Most benchmarks, including Market-1501, were collected over a short period, so each person wears the same clothes in every camera. Models trained on such data learn to rely heavily on clothing colour and texture, because these are the most discriminative cues available. In long-term scenarios, for example surveillance over several days, people change clothes, and these cues become misleading. Clothes-changing ReID asks the model to recognise a person from cues that do not depend on clothes, such as body shape, hairstyle, face and gait.

1.3 Objectives of this study

1. Understand the two selected papers in depth and explain how each method works.
2. Re-implement both methods and train them locally, following the official settings as closely as the available hardware allows.
3. Understand the datasets and the evaluation protocols used by each paper.
4. Compare the reproduced results with the published results, and compare the two methods under a common clothes-changing benchmark.

1.4 Summary of findings

- TransReID was reproduced almost exactly on Market-1501: 95.07% Rank-1 and 88.89% mAP, against 95.2% and 88.9% reported.
- CAL was reproduced on PRCC within about 2 points: 53.26% Rank-1 and 54.93% mAP in the clothes-changing setting, against 55.2% and 55.8% reported.
- CAL improves over the same network trained without it by +7.56 Rank-1, +9.18 mAP and +15.92 mINP in the clothes-changing setting.
- TransReID, although strong in the standard setting, loses about 50 points of mAP when clothes change and stays below CAL. The training objective matters more than the backbone for clothes changes.

1.5 Structure of the report

Section 2 introduces the problem formulation, evaluation metrics and common building blocks. Section 3 describes the datasets. Sections 4 and 5 explain TransReID and CAL. Section 6 describes the experimental setup and implementation. Section 7 presents the results, Section 8 discusses them, and

Section 9 concludes. The appendices list all hyper-parameters, the full evaluation logs and the steps needed to reproduce the experiments.

2 Background

2.1 Problem formulation

A ReID dataset is split into a training set and a test set with disjoint identities. The test set is further split into a query set and a gallery set. A model f maps an image x to a feature vector f(x). For each query q, all gallery images g are ranked by the cosine distance $d(q, g) = 2 - 2 \cos(f(q), f(g))$ after L2 normalisation. Following the standard protocol, gallery images of the same identity taken by the same camera as the query are ignored, because matching them is trivial.

2.2 Evaluation metrics

- **Rank-k accuracy (CMC curve):** the percentage of queries for which at least one correct match appears among the top k ranked gallery images. Rank-1 is the most commonly reported value.
- **Mean average precision (mAP):** for each query, the average precision over all of its correct gallery matches, averaged over all queries. Unlike Rank-k, mAP rewards finding all correct matches, not only the first one.
- **Mean inverse negative penalty (mINP):** for each query, the number of correct matches divided by the rank of the hardest (last) correct match, averaged over queries. It measures how much effort is needed to find all instances of a person, and is reported here as an additional metric.

2.3 Common building blocks

Both papers build on a set of techniques that are standard in modern ReID:

- **ImageNet pre-training** of the backbone (ResNet-50 or ViT-B/16).
- **Last stride 1:** the down-sampling in the last stage of ResNet-50 is removed, giving a 24 x 8 (CAL: 24 x 12) feature map with more spatial detail.
- **Identity loss:** the identities of the training set are treated as classes and a classifier is trained with cross-entropy, optionally with label smoothing.
- **Triplet loss with batch-hard mining:** each batch contains P identities with K images each. For every anchor, the farthest positive and the closest negative in the batch are used, and the loss pushes the negative farther than the positive.
- **BNNeck:** a batch normalisation layer between the feature used by the triplet loss and the feature fed to the classifier, which lets the two losses work in compatible feature spaces.
- **Random erasing:** a random rectangle of the training image is replaced by the mean pixel value, which simulates occlusion.

3 Datasets

3.1 Market-1501

Market-1501 was collected in front of a supermarket at Tsinghua University with six cameras (five high-resolution and one low-resolution). Bounding boxes were produced by the DPM pedestrian detector, so they include realistic detection errors such as misalignment and partial bodies. The dataset contains 32,668 annotated boxes of 1,501 identities. All images are 128 x 64 pixels. File names encode the identity, camera, video sequence and frame, for example 0002_c1s1_000451_03.jpg.

The gallery also contains 2,798 *distractor* images (identity 0), which are false detections or people outside the dataset, and 3,819 *junk* images (identity -1), which are ignored during evaluation. Table 1 lists the statistics measured by our exploration script. They match the official numbers.

Table 1. Market-1501 statistics (measured on the Kaggle copy of the dataset).

Split	Images	Identities	Cameras	Images per identity
Train (bounding_box_train)	12,936	751	6	17.2
Query	3,368	750	6	4.5
Gallery (without junk)	15,913	750 + distractors	6	17.5 (excluding 2,798 distractors)

3.2 PRCC

PRCC (Person Re-identification under moderate Clothing Change) contains 33,698 indoor images of 221 identities captured by three cameras, A, B and C. Cameras A and B see each person in the same clothes, while camera C sees the same person on another day in different clothes. The dataset also provides contour sketches, which are not used here because CAL uses RGB images only.

The training set has 150 identities. Each identity has two outfits, one in cameras A and B and one in camera C, giving 300 clothes classes for CAL. The test set has 71 identities. Camera A forms the gallery. Camera B forms the *same-clothes* query set and camera C the *clothes-changing* query set. The 5,002 validation images are not used.

Table 2. PRCC statistics (RGB modality).

Split	Images	Identities	Cameras	Outfits per identity
Train	17,896	150	3 (A, B, C)	2 (300 in total)
Validation (not used)	5,002	n/a	3	n/a
Gallery (camera A)	3,384	71	1	1
Same-clothes query (camera B)	3,873	71	1	1
Clothes-changing query (camera C)	3,543	71	1	1

3.3 LTCC

The CAL paper also reports results on LTCC (Long-Term Cloth-Changing), which contains 17,138 images of 152 identities wearing 478 outfits, captured by 12 cameras. LTCC is distributed only after the authors receive a signed data release agreement, and it was not available within the time frame of

this study. The implementation supports LTCC, including its clothes-changing protocol, so it can be added later without code changes.

3.4 Evaluation protocols

Table 3. Evaluation protocols used in this report.

Dataset	Setting	Query	Gallery	Gallery images ignored
Market-1501	standard	query	bounding_box_test	same identity and same camera; junk (id -1)
PRCC	same clothes	camera B	camera A	same identity and same camera (none occur)
PRCC	clothes-changing	camera C	camera A	same identity and same camera (none occur)
LTCC	clothes-changing	query	test	same identity with same camera or same clothes

All results use single-query evaluation without re-ranking. As in the official CAL code, the full PRCC gallery is used, rather than the single-shot protocol that samples one gallery image per identity.

[Figure placeholder: samples_market.png and samples_prcc.png from the final Kaggle run]

Figure 1. Example images. Top: one Market-1501 identity seen by different cameras. Bottom: one PRCC identity in its two outfits (cameras A and B versus camera C).

4 Paper 1: TransReID (ICCV 2021)

He, S., Luo, H., Wang, P., Wang, F., Li, H. and Jiang, W. TransReID: Transformer-based Object Re-Identification. ICCV 2021.

4.1 Motivation

Convolutional networks process images through local receptive fields and repeated down-sampling. The authors argue that this has two drawbacks for ReID. First, the network tends to focus on a few small discriminative regions instead of the whole body. Second, fine-grained details are lost by down-sampling. A vision transformer (ViT) uses self-attention between all image patches from the first layer, so it can model long-range relations across the whole body without down-sampling. TransReID adapts ViT to ReID and adds two modules designed for this task.

4.2 Transformer baseline

The image is split into fixed-size patches, and each patch is linearly projected to a 768-dimensional token. A learnable [cls] token is prepended and learnable position embeddings are added. The sequence passes through the 12 transformer layers of ViT-B/16. The output [cls] token is the image feature: the triplet loss is applied to it, and after a BNNeck, the identity loss. The paper uses two modifications in all its main results:

- **Overlapping patches.** Instead of non-overlapping 16 x 16 patches, patches are taken with stride 12, so neighbouring patches share 4 pixels. For a 256 x 128 image this gives 21 x 11 = 231 patches instead of 16 x 8 = 128, which preserves local structure around patch borders. The position embeddings of the pre-trained model are bilinearly interpolated to the new 21 x 11 grid. The method with this setting is denoted TransReID* in the paper.
- **Pre-training on ImageNet-21k**, followed by fine-tuning on ImageNet-1k.

4.3 Side Information Embedding (SIE)

Camera and viewpoint differences cause a systematic change in appearance that is not related to identity. SIE encodes this side information as learnable embeddings, one vector per camera (and optionally per viewpoint), and adds it to every token together with the position embedding, scaled by a coefficient (3.0 in the paper). The network can then learn to factor out camera-specific appearance. Market-1501 and PRCC provide camera labels but no viewpoint labels, so only the camera embedding is used.

4.4 Jigsaw Patch Module (JPM)

The last transformer layer is duplicated into two parallel branches. The *global branch* processes the full token sequence and outputs the global feature. The *jigsaw branch* first rearranges the patch tokens: a **shift** operation moves the first m = 5 patches to the end of the sequence, and a **patch shuffle** interleaves the sequence into k = 2 groups. The rearranged sequence is then divided into 4 parts. Each part, together with the shared [cls] token, passes through the jigsaw layer and produces one local feature. Because each local feature sees patches from different body regions, the model learns features that are robust to misalignment and that cover the whole body. At test time, the global feature and the four local features (each divided by 4) are concatenated into one 3,840-dimensional descriptor.

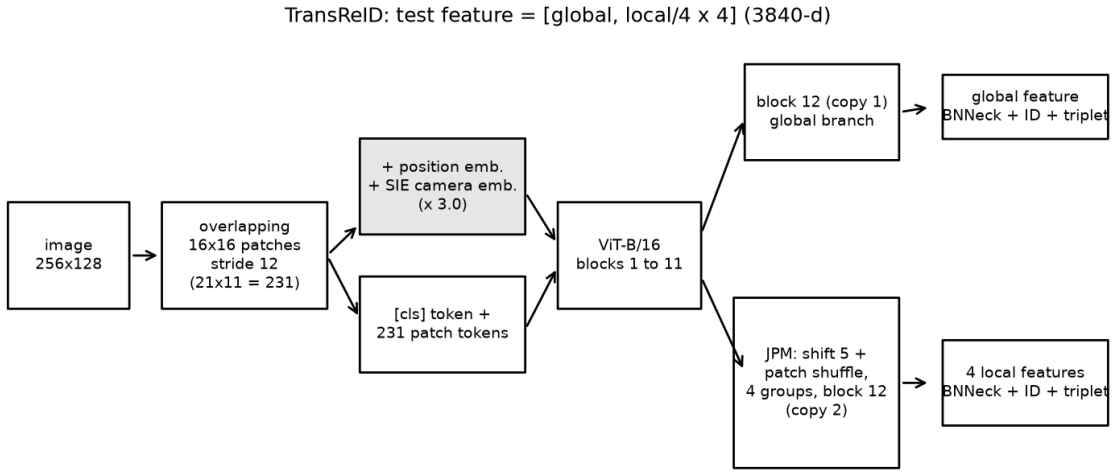


Figure 2. TransReID as implemented in this study (ViT-B/16, stride 12, SIE and JPM).

4.5 Training objective

Each of the five features (one global, four local) has its own BNNeck and identity classifier, and each uses a triplet loss. The total loss gives equal weight to the global branch and to the average of the local branches:

$$L = 0.5\,[L_{ID}(f_g) + L_T(f_g)] + 0.5 \times \frac{1}{4} \sum_{j=1}^{4} [L_{ID}(f_j) + L_T(f_j)]$$

Following the official configuration, the identity loss is plain cross-entropy (no label smoothing) and the triplet loss uses the soft-margin form $\log(1 + \exp(d_{ap} - d_{an}))$, with batch-hard mining.

Table 4. TransReID training settings (official configs/Market/vit_transreid_stride.yml).

Setting	Value
Backbone	ViT-B/16, ImageNet-21k pre-trained, ImageNet-1k fine-tuned
Input size / patch stride	256 x 128 / 12 (231 patches)
SIE coefficient / JPM	3.0 (camera only) / shift 5, shuffle groups 2, 4 parts
Batch	64 = 16 identities x 4 images
Augmentation	horizontal flip, padding 10 + random crop, random erasing (p = 0.5); mean and std 0.5
Optimiser	SGD, momentum 0.9, base lr 0.008 (bias lr x 2), weight decay 1e-4
Schedule	120 epochs, 5 linear warm-up epochs, cosine decay
Losses	cross-entropy + soft-margin batch-hard triplet, drop path 0.1
Test feature	global + 4 local features (before BNNeck), no flip, no re-ranking

5 Paper 2: CAL (CVPR 2022)

Gu, X., Chang, H., Ma, B., Bai, S., Shan, S. and Chen, X. Clothes-Changing Person Re-identification with RGB Modality Only. CVPR 2022.

5.1 Motivation

Earlier clothes-changing methods add extra modalities to obtain clothes-independent cues: contour sketches, body silhouettes, human pose or 3D body shape. These require extra models at training or test time and increase computation. CAL instead asks whether clothes-irrelevant features can be mined from the original RGB images alone. The key idea is to train the network so that its features can no longer tell apart the different outfits of the same person.

5.2 Clothes labels

CAL uses a *clothes label* for every training image, which is a finer version of the identity label: each outfit of each person is one class. In PRCC every training identity has two outfits, so there are 300 clothes classes for 150 identities. On a standard dataset such as Market-1501, each person has only one outfit, so clothes labels are the same as identity labels.

5.3 Clothes classifier

A cosine-similarity classifier predicts the clothes class from the L2-normalised feature. It is trained with cross-entropy on the scaled cosine similarity (scale s = 16, that is, temperature 1/16). The gradient of this loss is **not** propagated into the backbone: only the clothes classifier is updated, so it becomes a probe of how much clothing information the current features contain.

5.4 Clothes-based adversarial loss

The backbone is then trained against the clothes classifier. For an image with feature f, identity i and outfit c, let S+ be all outfits of identity i (the positive clothes classes) and S- all outfits of other identities. With w_k the normalised weight of clothes class k:

L_CA = - sum over k in S+ of q(k) log [exp(s cos(f, w_k)) / (exp(s cos(f, w_k)) + sum over j in S- of exp(s cos(f, w_j)))]

Here q(k) = (1 - e) for the true outfit plus e/|S+| spread over all outfits of the same person, with e = 0.1. Each positive outfit competes only against the outfits of *other* people, never against the other outfits of the same person. The loss therefore pulls the feature towards all outfits of the same person at once, and pushes it away from other people. When it is minimised, the outfits of one person become indistinguishable to the clothes classifier, and the remaining identity information has to come from cues that do not depend on clothes.

5.5 Training procedure

Training alternates between the two players. For the first 25 epochs only the identity loss is used, so the network first learns identity features. From epoch 26 onwards, every iteration (1) updates the clothes classifier with the clothes loss on detached features, and (2) updates the backbone and identity classifier with the identity loss plus L_CA. The identity loss uses a linear classifier with label-smoothed cross-entropy. The paper reports that the optional triplet loss is not needed.

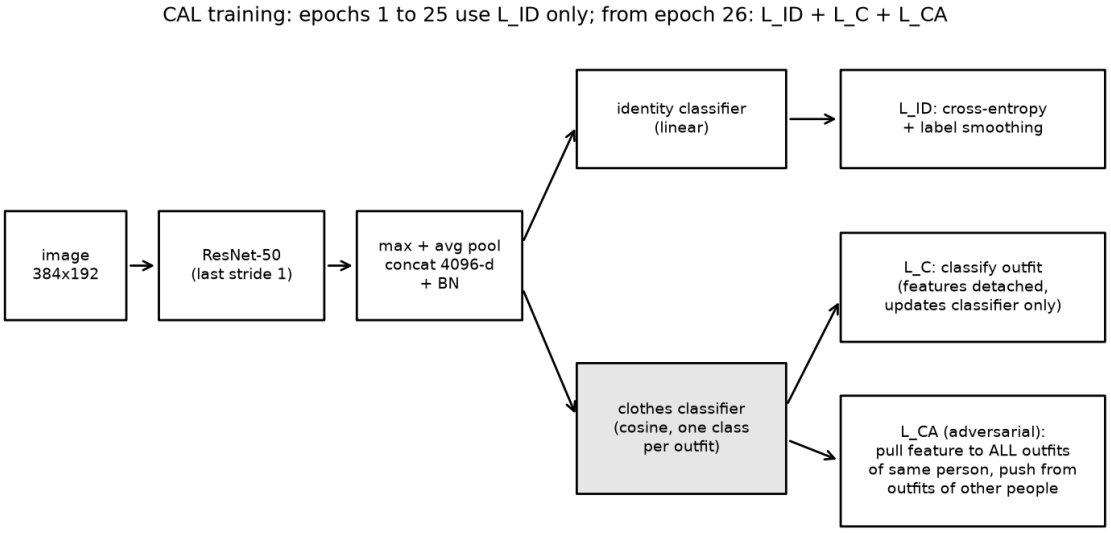


Figure 3. CAL as implemented in this study. The shaded clothes classifier is trained adversarially against the backbone.

Table 5. CAL training settings (official Simple-CCReID code, configs/res50_cels_cal.yaml).

Setting	**Value**
Backbone	ResNet-50 (ImageNet), last stride 1
Feature	concatenation of global max pooling and global average pooling (4,096-d) + BN
Input size / batch	384 x 192 / 32 = 4 identities x 8 images
Augmentation	random crop (enlarge by 1/8, p = 0.5), horizontal flip, random erasing (p = 0.5)
Identity loss	linear classifier, cross-entropy with label smoothing 0.1
Clothes classifier	cosine classifier, CosFace with scale 16 and margin 0
Adversarial loss	L_CA with scale 16, e = 0.1, active from epoch 26
Optimiser	Adam, lr 3.5e-4, weight decay 5e-4 (clothes classifier: constant lr)
Schedule	60 epochs, lr x 0.1 at epochs 20 and 40
Test feature	4,096-d BN feature, sum with horizontally flipped image, L2-normalised

6 Experimental Setup

6.1 Hardware and software

All experiments ran on Kaggle notebooks with two NVIDIA Tesla T4 GPUs (16 GB each), PyTorch 2.11 with CUDA 12.8, torchvision and timm. Kaggle limits a single session to 12 hours and provides about 30 GPU hours per week.

6.2 Implementation

Both methods were re-implemented in a single notebook. The code was written from the papers and checked line by line against the official repositories (damo-cv/TransReID and guxinqian/Simple-CCReID), including their configuration files. The pre-trained weights are the same as in the papers: ViT-B/16 pre-trained on ImageNet-21k and fine-tuned on ImageNet-1k (timm model vit_base_patch16_224.orig_in21k_ft_in1k, identical to the jx_vit_base_p16_224 weights used by TransReID), and the torchvision ImageNet ResNet-50.

To work within the 12-hour session limit, the training loop saves a full checkpoint after every epoch, resumes automatically from it, and stops cleanly before the limit. Each experiment is selected with a single switch in the notebook. A quick test mode runs every experiment for a few iterations before the real runs.

6.3 Differences from the official implementations

- **Mixed precision (FP16)** was used for speed on the T4 GPUs. The loss terms of CAL that use exponentials are computed in FP32.
- **TransReID used both GPUs** through data parallelism. Batch normalisation statistics are therefore computed over 32 images per GPU instead of 64.
- **CAL uses one Adam optimiser with two parameter groups** (backbone and clothes classifier), instead of two optimisers. The adversarial term uses the classifier weights without gradient, so the clothes classifier is updated only by its own loss, as in the original method. The only difference is that both updates happen in the same step instead of one after the other.
- **Random erasing** fills the erased area with zero after normalisation, which corresponds to the mean pixel value.

As in both official implementations, the model is evaluated on the test set during training (every 20 epochs for TransReID and every 5 epochs for CAL), and the best evaluation is reported. This follows the protocol of the papers, but it slightly favours the reported numbers, since no separate validation set is used for model selection.

6.4 Experiments

Table 6. Experiments run on Kaggle (2 x Tesla T4).

Experiment	Method	Train / test data	Epochs	Time per epoch	Total time
transreid_market	TransReID*	Market-1501	120	123 s	4.3 h
cal_prcc	CAL	PRCC	60	125 s	2.3 h
baseline_prcc	CAL network without L_C and L_CA	PRCC	60	122 s	2.3 h
transreid_prcc	TransReID*	PRCC	120	170 s	5.8 h

The first three experiments ran in one long session. Its result files were lost when Kaggle ended the interactive session, so their metrics were taken from the complete training log, which records every evaluation (Appendix B). The TransReID on PRCC experiment ran as a background job with all output files kept.

7 Results

7.1 TransReID on Market-1501

Table 7. TransReID on Market-1501: reproduction against the paper (%).

Method	Rank-1	Rank-5	Rank-10	mAP	mINP
TransReID* paper (ViT-B/16, stride 12, 256 x 128)	95.2	n/a	n/a	88.9	n/a
TransReID paper (ViT-B/16, stride 16)	95.0	n/a	n/a	88.2	n/a
TransReID* (ours, epoch 120)	95.07	98.19	98.96	88.89	68.58

The reproduction matches the paper almost exactly: 0.13 points lower in Rank-1 and 0.01 points lower in mAP. Figure 4 shows that most of the accuracy is reached early (93.38% Rank-1 after 20 epochs). The cosine schedule then adds about 4 points of mAP in the remaining 100 epochs. The best result is at the last epoch, so no early stopping was needed.

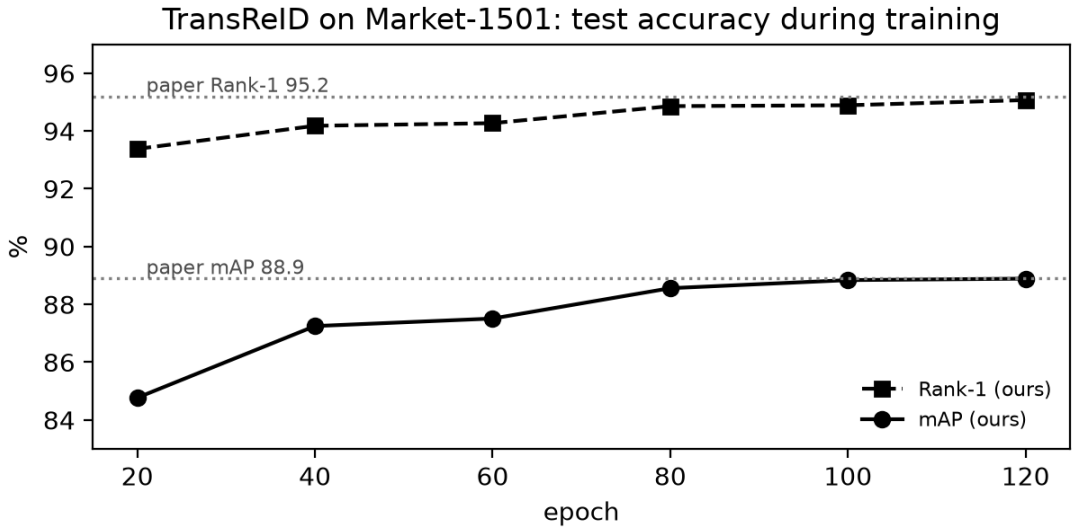


Figure 4. TransReID on Market-1501: Rank-1 and mAP on the test set during training. Dotted lines are the values reported in the paper.

7.2 CAL on PRCC

Table 8. CAL on PRCC: reproduction against the paper (%).

Method	Same clothes R1	Same clothes mAP	Clothes-changing R1	Clothes-changing mAP	Clothes-changing mINP
CAL paper	100	99.8	55.2	55.8	n/a
CAL (ours, epoch 55)	100.00	99.73	53.26	54.93	38.18

In the same-clothes setting the reproduction equals the paper. In the clothes-changing setting it is 1.94 points lower in Rank-1 and 0.87 points lower in mAP. This gap is within the run-to-run variation we observed: between epochs 35 and 60, with the learning rate already reduced, clothes-changing Rank-1 moved between 52.19% and 53.82% and mAP between 53.57% and 54.93%. The test set has only 71 identities, so a single run varies by about one point. Mixed precision and the merged optimiser step (Section 6.3) may also contribute.

7.3 Effect of the clothes-based adversarial loss

The baseline uses exactly the same network, data and schedule as CAL but is trained with the identity loss only. This isolates the contribution of CAL. Table 9 also lists the corresponding ablation from the CAL paper.

Table 9. Ablation on PRCC: baseline against CAL (%). Paper values from Table 3 of the CAL paper.

Method	Same clothes R1	Same clothes mAP	Clothes-changing R1	Clothes-changing mAP
Baseline, paper	99.8	97.9	45.6	43.3
Baseline, ours (epoch 45)	99.77	97.97	45.70	45.75
CAL, paper	100	99.8	55.2	55.8
CAL, ours (epoch 55)	100.00	99.73	53.26	54.93
Gain from CAL, ours	+0.23	+1.76	+7.56	+9.18
Gain from CAL, paper	+0.2	+1.9	+9.6	+12.5

Our baseline matches the paper's baseline closely (45.70% against 45.6% Rank-1), which confirms that the data loading, augmentation and evaluation protocol are the same. CAL then adds 7.56 points of Rank-1 and 9.18 points of mAP. The paper reports a slightly larger gain, but the direction and size of the effect are reproduced. CAL also improves mINP from 22.26% to 38.18%, so the hardest correct matches are found much earlier in the ranking.

Figure 5 shows when this happens. Before epoch 26, CAL and the baseline behave similarly, since both use only the identity loss. Once the clothes classifier and the adversarial loss are switched on, the training loss jumps from 0.90 to 5.38, and within five epochs the clothes-changing mAP of CAL rises from 42.81% to 51.47%. The baseline improves only slowly over the same period.

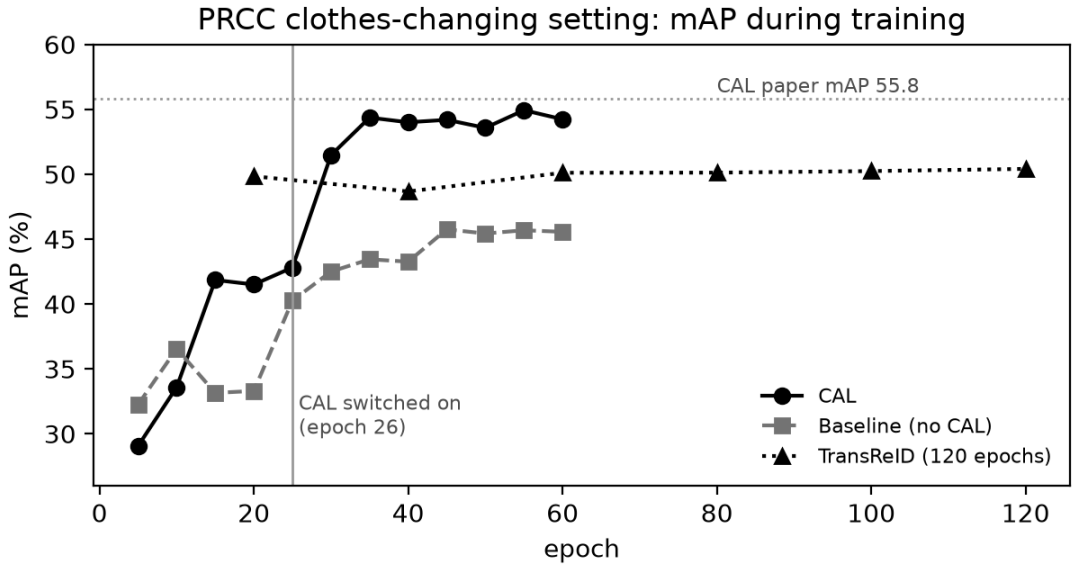


Figure 5. Clothes-changing mAP on PRCC during training for CAL, the baseline and TransReID. The vertical line marks the epoch where the clothes-based adversarial loss is switched on.

7.4 Comparison of the two papers on clothes-changing data

TransReID was designed for the standard setting and was not evaluated on clothes-changing data in its paper. To compare the two methods directly, TransReID* was trained on PRCC with exactly its Market-1501 settings.

Table 10. All methods on PRCC (%). Best evaluation of each run.

Method	Backbone	Same clothes R1	Same clothes mAP	Clothes-changing R1	Clothes-changing mAP	Clothes-changing mINP
Baseline	ResNet-50	99.77	97.97	45.70	45.75	22.26
TransReID*	ViT-B/16	99.97	99.12	48.60	50.41	34.04
CAL	ResNet-50	100.00	99.73	53.26	54.93	38.18

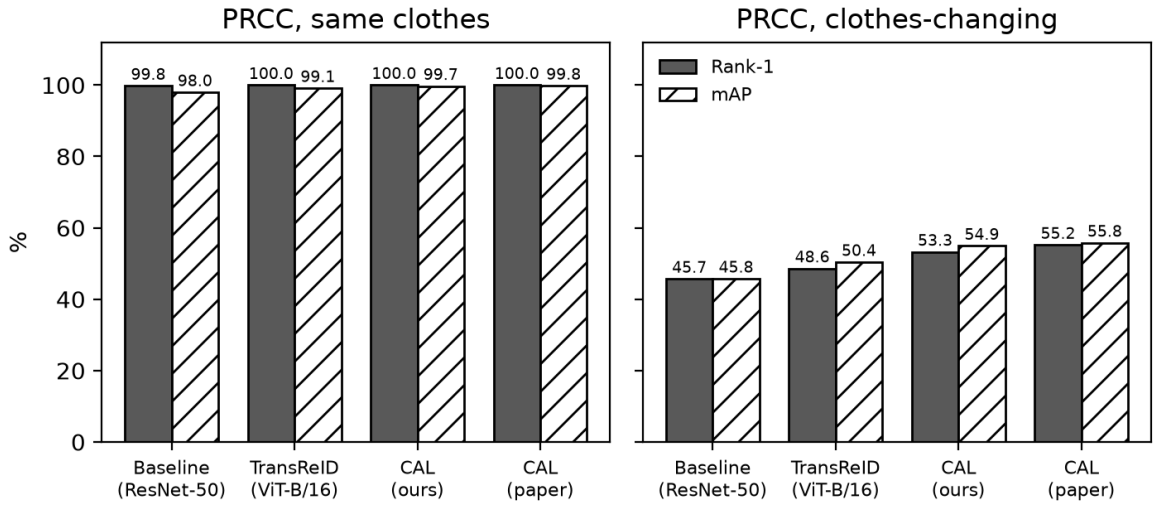


Figure 6. Rank-1 and mAP on PRCC for both settings. When clothes stay the same all methods are nearly perfect; when clothes change, CAL is clearly the best.

In the same-clothes setting all three methods are nearly perfect. In the clothes-changing setting, TransReID is better than the ResNet-50 baseline by 2.90 points of Rank-1 and 4.66 points of mAP, but CAL is better than TransReID by 4.66 points of Rank-1 and 4.52 points of mAP, although CAL uses the smaller ResNet-50 backbone. Interestingly, the CAL paper also reports a ResNet-50 variant trained with a triplet loss, which reaches 48.6% Rank-1 and 49.7% mAP. This is almost identical to TransReID, which also uses identity and triplet losses.

7.5 Computational cost

Table 11. Training cost on 2 x Tesla T4.

Experiment	GPUs used	Images per epoch	Time per epoch	Test time	Total training time
TransReID*, Market-1501	2	12,936	123 s	70 s	4.3 h (120 epochs)
TransReID*, PRCC	2	17,896	170 s	40 to 63 s	5.8 h (120 epochs)
CAL, PRCC	1	17,896	125 s	60 s	2.3 h (60 epochs)
Baseline, PRCC	1	17,896	122 s	60 s	2.3 h (60 epochs)

TransReID is the more expensive model: about 86 million parameters in the ViT-B/16 backbone and 232 tokens per image, compared with about 24 million parameters for the ResNet-50 used by CAL. It also needs twice as many epochs. CAL adds almost no cost: the clothes classifier is one small matrix, and its training step is the same length as the baseline's.

7.6 Qualitative results

[Figure placeholder: retrieval_transreid_prcc.png and retrieval_cal_prcc.png from the final Kaggle run]

Figure 7. Retrieval examples on PRCC in the clothes-changing setting: for the same queries, the top 10 gallery images returned by TransReID and by CAL. A thick solid frame marks a correct match and a dashed frame a wrong one.

8 Discussion

8.1 Reproducibility

Both papers proved reproducible from their public code and configuration files. TransReID matched to within 0.13 points, and CAL to within about 2 points on a small test set. Three details were important to match: TransReID uses a soft-margin triplet loss and no label smoothing in its final configuration, which differs from many earlier ReID baselines; CAL uses a linear identity classifier with label smoothing but a cosine clothes classifier; and PRCC is evaluated on the full gallery. Reproducing the CAL paper's baseline was a useful check, since it confirms the shared parts of the pipeline before the method itself is compared.

8.2 Why a strong standard model fails when clothes change

TransReID reaches 99.97% Rank-1 on same-clothes PRCC queries but only 48.60% on clothes-changing queries. Its training loss on PRCC falls to about 0.01, so it separates the 150 training identities almost perfectly. In PRCC, two thirds of each person's training images show the same outfit, so the easiest way to separate identities is by clothing, and nothing in the objective discourages this. The larger backbone and global attention of TransReID give a modest improvement over the ResNet-50 baseline, but they do not change what the model learns to look at.

CAL changes the objective instead. By making the outfits of one person indistinguishable to an adversarial classifier, it removes the shortcut, and the network must rely on other cues. This gives the largest improvement of all methods compared here, with the smaller backbone and almost no extra cost. For clothes-changing ReID, the training objective therefore matters more than the network architecture.

8.3 Practical considerations

- **Annotation.** CAL needs clothes labels for the training set. These exist in clothes-changing datasets but are extra work for new data. TransReID needs camera labels for SIE, which any multi-camera system provides.
- **Compatibility.** CAL is a loss function and does not depend on the backbone. Combining CAL with the TransReID backbone is a natural extension that was not tested here.
- **Compute.** TransReID needed about 2.5 times more GPU time than CAL on PRCC.

8.4 Limitations

- Each experiment was run once. The variation of about one point between evaluations suggests that the differences between the two papers' results are meaningful, but the small gaps to the published numbers are within run-to-run noise.
- Model selection used the test set, following both papers.
- LTCC, the second clothes-changing dataset of the CAL paper, could not be obtained in time.
- The hardware (T4 GPUs with mixed precision) differs from the original papers (V100 GPUs).

9 Conclusion

This study re-implemented and trained two recent person re-identification methods and reproduced their main results: TransReID on Market-1501 (95.07% Rank-1, 88.89% mAP, against 95.2% and 88.9%) and CAL on PRCC (53.26% Rank-1, 54.93% mAP in the clothes-changing setting, against 55.2% and 55.8%). Our reproduction of the CAL paper's baseline and its ablation confirmed that the clothes-based adversarial loss adds about 8 to 9 points in the clothes-changing setting.

Evaluating TransReID on clothes-changing data showed that a model which is excellent when clothes stay the same can lose about half of its accuracy when they change. The transformer backbone improves on a ResNet-50 baseline, but CAL, with a smaller backbone and a different objective, improves much more. Future work includes evaluating on LTCC, running several seeds per experiment, and combining the CAL objective with a transformer backbone such as TransReID.

References

[1] He, S., Luo, H., Wang, P., Wang, F., Li, H. and Jiang, W. TransReID: Transformer-based Object Re-Identification. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2021. arXiv:2102.04378.

[2] Gu, X., Chang, H., Ma, B., Bai, S., Shan, S. and Chen, X. Clothes-Changing Person Re-identification with RGB Modality Only. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022. arXiv:2204.06890.

[3] Zheng, L., Shen, L., Tian, L., Wang, S., Wang, J. and Tian, Q. Scalable Person Re-identification: A Benchmark. In ICCV, 2015.

[4] Yang, Q., Wu, A. and Zheng, W.-S. Person Re-identification by Contour Sketch under Moderate Clothing Change. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019.

[5] Qian, X., Wang, W., Zhang, L., Zhu, F., Fu, Y., Xiang, T., Jiang, Y.-G. and Xue, X. Long-Term Cloth-Changing Person Re-identification. In ACCV, 2020.

[6] Dosovitskiy, A. et al. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In ICLR, 2021.

[7] He, K., Zhang, X., Ren, S. and Sun, J. Deep Residual Learning for Image Recognition. In CVPR, 2016.

[8] Luo, H., Gu, Y., Liao, X., Lai, S. and Jiang, W. Bag of Tricks and a Strong Baseline for Deep Person Re-identification. In CVPR Workshops, 2019.

[9] Hermans, A., Beyer, L. and Leibe, B. In Defense of the Triplet Loss for Person Re-Identification. arXiv:1703.07737, 2017.

[10] Ye, M., Shen, J., Lin, G., Xiang, T., Shao, L. and Hoi, S. C. H. Deep Learning for Person Re-identification: A Survey and Outlook. IEEE TPAMI, 2021.

[11] Zhong, Z., Zheng, L., Kang, G., Li, S. and Yang, Y. Random Erasing Data Augmentation. In AAAI, 2020.

[12] Official code: github.com/damo-cv/TransReID and github.com/guxinqian/Simple-CCReID.

Appendix A: Evaluation Logs

All test-set evaluations recorded during training (%). SC = same clothes, CC = clothes-changing.

Table 12. TransReID* on Market-1501.

Epoch	Rank-1	Rank-5	Rank-10	mAP	mINP
20	93.38	97.57	98.55	84.77	60.75
40	94.18	97.95	98.81	87.25	65.52
60	94.27	98.04	98.99	87.51	66.04
80	94.86	98.22	98.93	88.56	67.79
100	94.89	98.25	98.90	88.84	68.54
120	95.07	98.19	98.96	88.89	68.58

Table 13. CAL on PRCC.

Epoch	SC R1	SC mAP	CC R1	CC R5	CC R10	CC mAP	CC mINP
5	94.89	86.33	34.10	40.76	44.23	29.06	8.53
10	96.54	89.52	36.55	42.93	46.32	33.59	13.31
15	99.33	95.09	45.39	50.83	53.46	41.85	14.05
20	97.19	92.79	45.55	51.37	55.04	41.50	13.07
25	99.35	96.47	45.98	51.03	53.60	42.81	13.25
30	100.00	99.43	50.55	55.77	57.95	51.47	34.32
35	100.00	99.48	53.32	58.54	61.11	54.36	36.30
40	100.00	99.63	53.82	57.32	58.62	54.00	36.84
45	100.00	99.66	52.72	56.51	58.23	54.19	36.74
50	100.00	99.66	52.19	57.86	60.32	53.57	35.93
55	100.00	99.73	53.26	57.66	59.75	54.93	38.18
60	100.00	99.75	52.81	56.59	58.85	54.23	36.98

Table 14. Baseline (no CAL) on PRCC.

Epoch	SC R1	SC mAP	CC R1	CC R5	CC R10	CC mAP	CC mINP
5	96.15	86.65	38.92	46.43	49.51	32.25	10.23
10	98.06	94.03	39.60	43.66	45.81	36.55	15.13
15	98.58	94.33	34.04	38.27	40.84	33.15	13.02
20	98.37	94.37	35.85	43.16	46.18	33.30	9.16
25	99.74	96.77	42.79	46.99	49.28	40.26	13.86
30	99.64	96.98	44.96	50.30	52.81	42.50	16.56
35	99.69	97.78	44.20	48.94	52.07	43.44	17.57
40	99.51	97.57	43.66	48.86	52.02	43.26	18.84
45	99.77	97.97	45.70	50.64	54.05	45.75	22.26
50	99.82	98.01	46.54	51.03	53.66	45.43	20.80
55	99.69	97.93	46.88	51.51	54.11	45.68	22.29
60	99.74	97.97	46.57	51.65	54.22	45.56	19.60

Table 15. TransReID* on PRCC.

Epoch	SC R1	SC mAP	CC R1	CC R5	CC R10	CC mAP	CC mINP
20	100.00	99.10	48.66	54.39	56.62	49.84	32.73
40	99.97	99.29	46.63	53.82	56.48	48.67	33.71
60	99.95	99.12	49.03	54.78	58.26	50.11	33.36
80	99.95	99.19	48.91	54.76	57.92	50.12	34.20
100	99.97	99.11	48.74	54.25	56.70	50.24	33.79
120	99.97	99.12	48.60	54.61	57.24	50.41	34.04

Appendix B: Reproducing the Experiments

All code is in one Kaggle notebook (file reid_kaggle_notebook.py, nine cells). The datasets are attached as Kaggle inputs: Market-1501 (iiierie/market-1501-v15-09-15) and PRCC (lyqassf/prccdata1).

1. Create a Kaggle notebook with GPU T4 x2 and internet enabled, and attach both datasets.
2. Paste the nine cells. Cell 1 writes the shared library, Cells 3 to 5 define the dataset exploration, TransReID and CAL, Cell 6 runs the selected experiment, and Cells 7 to 9 create the result tables, figures and a zip file.
3. In Cell 2, set JOB to one of transreid_market, cal_prcc, baseline_prcc or transreid_prcc, and QUICK to False.
4. Run the notebook with Save Version, Save and Run All. If a run stops, attach the previous version as an input and run again: training resumes from the last saved epoch.